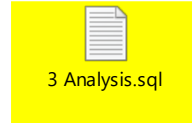
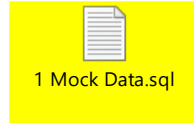


Data Analyst – Internship

Task 6: Sales Trend Analysis Using Aggregations

Tool: PostgreSQL



Objective:

- Monthly Revenue
- Monthly Order Volume

We'll use:

- `EXTRACT (MONTH FROM order_date)`
- `SUM ( )`
- `COUNT (DISTINCT order_id)`
- `GROUP BY`
- `ORDER BY`

Step: 1 – Monthly Revenue Analysis

ecommerce/postgres@PostgreSQL 18

Query Query History

```
1  --Monthly revenue analysis
2  SELECT
3      EXTRACT(YEAR FROM order_date) AS year,
4      EXTRACT(MONTH FROM order_date) AS month,
5      SUM(total_amount) AS monthly_revenue
6  FROM orders
7  WHERE status = 'Completed'
8  GROUP BY year, month
9  ORDER BY year, month;
```

Data Output Messages Notifications

	year numeric	month numeric	monthly_revenue numeric
1	2023	1	19.99
2	2023	2	549.96
3	2023	3	2808.97
4	2023	4	379.94
5	2023	5	1429.47
6	2023	6	149.99
7	2023	7	649.96
8	2023	8	799.94
9	2023	9	2039.47
10	2023	10	549.98
11	2024	1	59.98
12	2024	2	999.97
13	2024	3	849.93

Total rows: 35 of 35 Query complete 00:00:00.086

Step: 2 – Revenue by Month

ecommerce/postgres@PostgreSQL 18

Query Query History

```
11 --Revenue by month
12 SELECT
13     DATE(DATE_TRUNC('month', order_date)) AS month,
14     COUNT(DISTINCT order_id) AS total_orders,
15     SUM(total_amount) AS total_revenue
16 FROM orders
17 WHERE status = 'Completed'
18 AND order_date >= CURRENT_DATE - INTERVAL '12 months'
19 GROUP BY month
20 ORDER BY month;
```

Data Output Messages Notifications

	month date	total_orders bigint	total_revenue numeric
1	2025-03-01	2	519.93
2	2025-04-01	1	149.97
3	2025-05-01	3	1649.96
4	2025-06-01	2	129.95
5	2025-07-01	2	1999.49
6	2025-08-01	1	399.99
7	2025-09-01	2	359.95
8	2025-10-01	2	1339.46
9	2026-01-01	1	699.99
10	2026-02-01	1	59.98
11	2026-03-01	1	79.96
12	2026-04-01	1	149.99

Total rows: 13 of 13 Query complete 00:00:00.092

Step: 3 – Current year monthly trend

ecommerce/postgres@PostgreSQL 18

Query Query History

```
--Current year monthly trend
SELECT
    EXTRACT(MONTH FROM order_date) AS month,
    COUNT(DISTINCT order_id) AS total_orders,
    SUM(total_amount) AS total_revenue
FROM orders
WHERE status = 'Completed'
AND DATE_TRUNC('year', order_date) = DATE_TRUNC('year', CURRENT_DATE)
GROUP BY month
ORDER BY month;
```

Data Output Messages Notifications

	month numeric	total_orders bigint	total_revenue numeric
1	1	1	699.99
2	2	1	59.98
3	3	1	79.96
4	4	1	149.99
5	5	1	1299.50

Step: 4 – Previous year analysis

ecommerce/postgres@PostgreSQL 18

Query Query History

```
34 SELECT
35     EXTRACT(MONTH FROM order_date) AS month,
36     COUNT(DISTINCT order_id) AS total_orders,
37     SUM(total_amount) AS total_revenue
38 FROM orders
39 WHERE status = 'Completed'
40 AND DATE_TRUNC('year', order_date) =
41     DATE_TRUNC('year', CURRENT_DATE - INTERVAL '1 year')
42 GROUP BY month
43 ORDER BY month;
```

Data Output Messages Notifications

	month numeric	total_orders bigint	total_revenue numeric
1	1	1	849.98
2	2	1	119.96
3	3	2	519.93
4	4	1	149.97
5	5	3	1649.96
6	6	2	129.95
7	7	2	1999.49
8	8	1	399.99
9	9	2	359.95
10	10	2	1339.46

Step: 5 – Last 3 years trend

ecommerce/postgres@PostgreSQL 18

Query Query History

```
45 --Last 3 years trend
46 SELECT
47     EXTRACT(YEAR FROM order_date) AS year,
48     COUNT(DISTINCT order_id) AS total_orders,
49     SUM(total_amount) AS total_revenue
50 FROM orders
51 WHERE status = 'Completed'
52 AND order_date >= CURRENT_DATE - INTERVAL '3 years'
53 GROUP BY year
54 ORDER BY year;
```

Data Output Messages Notifications

	year numeric	total_orders bigint	total_revenue numeric
1	2023	13	8807.72
2	2024	16	6898.58
3	2025	17	7518.64
4	2026	5	2289.42

Step: 6 – Current quarter analysis

ecommerce/postgres@PostgreSQL 18

Query Query History

```
55
56 --Current quarter analysis
57 SELECT
58     COUNT(DISTINCT order_id) AS total_orders,
59     SUM(total_amount) AS total_revenue
60 FROM orders
61 WHERE status = 'Completed'
62 AND DATE_TRUNC('quarter', order_date) =
63     DATE_TRUNC('quarter', CURRENT_DATE);
64
```

Data Output Messages Notifications

	total_orders bigint	total_revenue numeric
1	3	839.93

Step: 7 – Rolling 6-month profit trend

ecommerce/postgres@PostgreSQL 18

Query Query History

```
65 --Rolling 6-month profit trend
66 SELECT
67     DATE(DATE_TRUNC('month', o.order_date)) AS month,
68     SUM(oi.quantity * oi.price) AS total_revenue,
69     SUM(oi.quantity * p.cost_price) AS total_cost,
70     SUM(oi.quantity * oi.price) - SUM(oi.quantity * p.cost_price) AS total_profit
71 FROM orders o
72 JOIN order_items oi ON o.order_id = oi.order_id
73 JOIN products p ON oi.product_id = p.product_id
74 WHERE o.status = 'Completed'
75 AND o.order_date >= CURRENT_DATE - INTERVAL '6 months'
76 GROUP BY month
77 ORDER BY month;
```

Data Output Messages Notifications

	month date	total_revenue numeric	total_cost numeric	total_profit numeric
1	2025-08-01	399.99	280.00	119.99
2	2025-09-01	359.95	220.00	139.95
3	2025-10-01	1339.46	1018.00	321.46
4	2026-01-01	699.99	520.00	179.99
5	2026-02-01	59.98	24.00	35.98
6	2026-03-01	79.96	40.00	39.96
7	2026-04-01	149.99	95.00	54.99
8	2026-05-01	1299.50	1000.00	299.50

Step: 8 – Year-to-date (YTD) Analysis

The screenshot shows a PostgreSQL query editor interface. At the top, the connection is set to 'ecommerce/postgres@PostgreSQL 18'. Below the connection bar is a toolbar with icons for file operations, query execution, and settings. The 'Query' tab is active, displaying a SQL query for a Year-to-Date (YTD) analysis. The query counts distinct orders and sums the total amount for completed orders from the current year onwards. Below the query editor, the 'Data Output' tab shows the results of the query in a table format.

```
79 --Year-to-Date (YTD) analysis
80 SELECT
81     COUNT(DISTINCT order_id) AS ytd_orders,
82     SUM(total_amount) AS ytd_revenue
83 FROM orders
84 WHERE status = 'Completed'
85 AND order_date >= DATE_TRUNC('year', CURRENT_DATE);
86
```

	ytd_orders bigint	ytd_revenue numeric
1	5	2289.42